PATRONES DE DISE?O

Gu?a te?rica de los patrones creacionales

Singleton ? Prototype ? Builder ? Factory Method ? Abstract Factory

Apunte de estudio en espa?ol basado en los temas de la playlist de TodoCode Academy y en los ejemplos Java presentes en este repositorio.

Conceptos, estructura, ventajas, l?mites y criterios de uso

C?mo leer esta gu?a

Los patrones de dise?o son soluciones reutilizables para problemas recurrentes de dise?o de software. No son c?digo para copiar de forma mec?nica: describen una estructura y una colaboraci?n entre objetos que se adapta al problema concreto.

Los cinco temas que se desarrollan pertenecen a los patrones creacionales. Todos ayudan a controlar c?mo se construyen los objetos, aunque cada uno resuelve una necesidad distinta.

Mapa r?pido

Patr?n	Pregunta que responde
Singleton	?C?mo garantizo una ?nica instancia compartida?
Prototype	?C?mo creo objetos a partir de un ejemplar existente?
Builder	?C?mo construyo un objeto complejo paso a paso?
Factory Method	?C?mo delego en subclases la elecci?n del producto?
Abstract Factory	?C?mo creo familias de productos relacionados?

Idea transversal

Una buena abstracci?n permite que el c?digo cliente dependa de contratos estables y que las decisiones de creaci?n puedan cambiar sin reescribir toda la l?gica de negocio. La flexibilidad tiene un costo: m?s tipos y m?s estructura.

1. Singleton

Prop?sito

Asegura que una clase tenga una sola instancia accesible desde un punto com?n. Es ?til cuando el sistema necesita representar un recurso ?nico dentro de un contexto, por ejemplo una configuraci?n compartida.

Estructura conceptual

La clase restringe la creaci?n directa, conserva una referencia est?tica a su instancia y ofrece un m?todo de acceso. El constructor suele ser privado. En los ejemplos Java del repositorio, Configuracion representa informaci?n centralizada.

Ventajas

Centraliza el acceso y evita crear accidentalmente varias copias de un recurso que debe ser ?nico.

L?mites y cuidados

Introduce estado global: cualquier parte puede depender de ?l, lo que dificulta aislar pruebas y entender el flujo de datos. En entornos concurrentes hay que resolver la inicializaci?n segura. La unicidad es por proceso/cargador, no necesariamente para toda una infraestructura distribuida.

Cu?ndo elegirlo

Cuando la unicidad sea un requisito real y su alcance est? claramente definido. No debe usarse solo para evitar pasar dependencias expl?citas.

2. Prototype

Prop?sito

Permite crear un objeto nuevo copiando un prototipo existente, en lugar de reconstruirlo campo por campo desde cero.

Estructura conceptual

Se define una operaci?n de clonaci?n en el tipo base o en una interfaz. Cada clase concreta sabe producir una copia de s? misma y el cliente solicita la copia al prototipo. Los pedidos de pizza del repositorio ilustran una jerarqu?a clonable.

Copia superficial y profunda

Una copia superficial duplica el objeto externo, pero conserva referencias a los mismos objetos internos. Una copia profunda replica tambi?n los objetos anidados que deban quedar independientes. La elecci?n depende de la mutabilidad que se permita compartir.

Ventajas y l?mites

Reduce la complejidad de crear objetos costosos o configurados. Exige definir qu? significa ?copiar? y puede ser fr?gil con referencias, recursos externos o invariantes complejas.

Cu?ndo elegirlo

Cuando crear desde cero sea costoso, haya muchas variantes configuradas o el cliente deba duplicar objetos sin acoplarse a sus clases concretas.

3. Builder

Prop?sito

Separa la construcci?n gradual de un objeto complejo de su representaci?n final. Resulta ?til con muchos par?metros opcionales o una secuencia de pasos.

Estructura conceptual

Un Builder declara operaciones de construcci?n y mantiene el objeto en formaci?n. Builders concretos producen representaciones espec?ficas. Un Director puede encapsular recetas repetibles; el cliente tambi?n puede coordinar los pasos. En los ejemplos, BuilderDirector organiza la construcci?n de una Casa.

Ventajas

Evita constructores con largas listas de par?metros, hace expl?citos los pasos y permite reutilizar secuencias para distintas representaciones. Tambi?n puede validar el resultado al finalizar.

L?mites y cuidados

A?ade clases y coordinaci?n. Para objetos simples, un constructor breve o un m?todo f?brica suele ser m?s claro. Defin? si el builder puede reutilizarse y qu? ocurre ante una construcci?n incompleta.

Cu?ndo elegirlo

Cuando el objeto tiene muchas partes, opciones, reglas de orden o distintas representaciones que comparten un proceso.

4. Factory Method

Prop?sito

Define un m?todo de creaci?n en una clase base y permite que las subclases decidan qu? clase concreta instanciar. Desacopla al cliente de la elecci?n del producto.

Estructura conceptual

El Producto declara la interfaz com?n; los Productos Concretos la implementan. El Creator declara el m?todo f?brica y puede incluir l?gica que usa el producto. Los Creadores Concretos sobrescriben el m?todo para devolver productos diferentes. En el repositorio, distintos creadores producen env?os por moto, correo o cami?n.

Colaboraci?n

El cliente trabaja con el Creator o el Producto abstracto. La subclase creadora decide qu? implementaci?n concreta crear. La l?gica com?n puede permanecer en el Creator, mientras cambia la decisi?n de instanciaci?n.

Ventajas y l?mites

Reduce el acoplamiento con clases concretas y facilita extender productos. Puede requerir una subclase creadora por variante. No es sin?nimo de cualquier m?todo est?tico que construya objetos: lo central es delegar la decisi?n en una especializaci?n del Creator.

Cu?ndo elegirlo

Cuando una clase conoce el trabajo que debe realizar con un producto, pero las subclases deben elegir qu? implementaci?n concreta se utiliza.

5. Abstract Factory

Prop?sito

Ofrece una interfaz para crear familias de productos relacionados o compatibles sin especificar sus clases concretas.

Estructura conceptual

La f?brica abstracta declara una operaci?n por tipo de producto. Cada f?brica concreta implementa esas operaciones para una familia coherente. Los productos comparten interfaces; el cliente recibe una f?brica y crea el conjunto mediante ella. En los ejemplos de ropa, f?bricas de outfit producen prendas coordinadas.

Ventajas

Mantiene la compatibilidad dentro de una familia y permite cambiar la familia completa en un punto. El cliente es independiente de los tipos concretos.

L?mites y cuidados

Agregar una nueva familia es directo, pero sumar un tipo de producto suele exigir cambiar la interfaz y todas las f?bricas concretas. Conviene cuando las familias y sus relaciones son parte estable del problema.

Cu?ndo elegirlo

Cuando el sistema deba operar con familias intercambiables de objetos que deben combinar entre s?, como temas visuales, proveedores o configuraciones de plataforma.

Comparaci?n y elecci?n

Patr?n	Qu? var?a	Escala
Singleton	N?mero y acceso a instancia	Una instancia compartida
Prototype	Forma de copiar	Objeto basado en otro
Builder	Pasos o representaci?n	Un objeto complejo
Factory Method	Subclase que elige producto	Un producto por creaci?n
Abstract Factory	Familia compatible	Varios productos relacionados

Preguntas para decidir

1. ?Necesito controlar una ?nica instancia? Considere Singleton y defina su alcance.
2. ?Ya tengo un objeto v?lido del que conviene obtener copias? Considere Prototype.
3. ?La construcci?n requiere muchos pasos u opciones? Considere Builder.
4. ?La subclase debe decidir qu? producto crear? Considere Factory Method.
5. ?Debo crear varios productos que pertenecen a una familia? Considere Abstract Factory.

Principios relacionados

Estos patrones pueden apoyar la inversi?n de dependencias: el c?digo de alto nivel usa abstracciones en vez de depender directamente de clases concretas. Tambi?n pueden reducir acoplamiento y facilitar la extensi?n. Aplicarlos no garantiza por s? solo SOLID; deben responder a una necesidad y seguir siendo f?ciles de comprender.

Fuentes y alcance

Playlist: TodoCode Academy - Patrones de dise?o. Los ejemplos se cotejaron con los directorios Java del repositorio: singleton, prototype, builder, factoryMethod y abstractFactory. Esta gu?a sintetiza teor?a para estudiar; no es una transcripci?n literal de los videos.